

35.3 The set-covering problem

The set-covering problem is an optimization problem that models many problems that require resources to be allocated. Its corresponding decision problem generalizes the NP-complete vertex-cover problem and is therefore also NP-hard. The approximation algorithm developed to handle the vertex-cover problem doesn't apply here, however. Instead, this section investigates a simple greedy heuristic with a logarithmic approximation ratio. That is, as the size of the instance gets larger, the size of the approximate solution may grow, relative to the size of an optimal solution. Because the logarithm function grows rather slowly, however, this approximation algorithm may nonetheless give useful results.

An instance $(X, \mathcal{F})$ of the *set-covering problem* consists of a finite set X and a family $\mathcal{F}$ of subsets of X , such that every element of X belongs to at least one subset in $\mathcal{F}$:

$$X = \bigcup_{S \in \mathcal{F}} S.$$

We say that a subfamily $\mathcal{C} \subseteq \mathcal{F}$ *covers* a set of elements U if

$$U \subseteq \bigcup_{S \in \mathcal{C}} S.$$

The problem is to find a minimum-size subfamily $\mathcal{C} \subseteq \mathcal{F}$ whose members cover all of X :

$$X = \bigcup_{S \in \mathcal{C}} S.$$

Figure 35.3 illustrates the set-covering problem. The size of $\mathcal{C}$ is the number of sets it contains, rather than the number of individual elements in these sets, since every subfamily $\mathcal{C}$ that covers X must contain all $|X|$ individual elements. In Figure 35.3, the minimum set cover has size 3.

The set-covering problem abstracts many commonly arising combinatorial problems. As a simple example, suppose that X represents a set of skills that are needed to solve a problem and that

you have a given set of people available to work on the problem. You wish to form a committee, containing as few people as possible, such that for every requisite skill in X , at least one member of the committee has that skill. The decision version of the set-covering problem asks whether a covering exists with size at most k , where k is an additional parameter specified in the problem instance. The decision version of the problem is NP-complete, as Exercise 35.3-2 asks you to show.

A greedy approximation algorithm

The greedy method in the procedure GREEDY-SET-COVER on the facing page works by picking, at each stage, the set S that covers the greatest number of remaining elements that are uncovered. In the example of Figure 35.3, GREEDY-SET-COVER adds to $\mathcal{C}$, in order, the sets S_1 , S_4 , and S_5 , followed by either S_3 or S_6 .

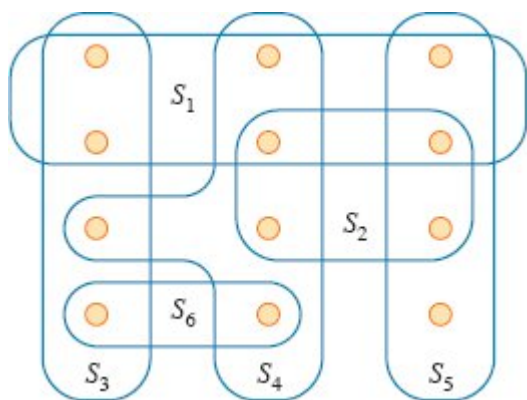


Figure 35.3 An instance $(X, \mathcal{F})$ of the set-covering problem, where X consists of the 12 tan points and $\mathcal{F} = \{S_1, S_2, S_3, S_4, S_5, S_6, S_4, S_5\}$. Each set $S_i \in \mathcal{F}$ is outlined in blue. A minimum-size set cover $\mathcal{C} = \{S_3, S_4, S_5\}$, with size 3. The greedy algorithm produces a cover of size 4 by selecting either the sets S_1, S_4, S_5 , and S_3 or the sets S_1, S_4, S_5 , and S_6 , in order.

GREEDY-SET-COVER $(X, \mathcal{F})$

1 $U_0 = X$

2 $\mathcal{C} = \emptyset$

3 $i = 0$

4

```

while  $U_i \neq \emptyset$ 
5   select  $S \in \mathcal{F}$  that maximizes  $|S \cap U_i|$ 
6    $U_{i+1} = U_i - S$ 
7    $\mathcal{C} = \mathcal{C} \cup \{S\}$ 
8    $i = i + 1$ 
9 return  $\mathcal{C}$ 

```

The greedy algorithm works as follows. At the start of each iteration, U_i is a subset of X containing the remaining uncovered elements, with the initial subset U_0 containing all the elements in X . The set $\mathcal{C}$ contains the subfamily being constructed. Line 5 is the greedy decision-making step, choosing a subset S that covers as many uncovered elements as possible (breaking ties arbitrarily). After S is selected, line 6 updates the set of remaining uncovered elements, denoting it by U_{i+1} , and line 7 places S into $\mathcal{C}$. When the algorithm terminates, $\mathcal{C}$ is a subfamily of $\mathcal{F}$ that covers X .

Analysis

We now show that the greedy algorithm returns a set cover that is not too much larger than an optimal set cover.

Theorem 35.4

The procedure GREEDY-SET-COVER run on a set X and family of subsets $\mathcal{F}$ is a polynomial-time $O(\lg X)$ -approximation algorithm.

Proof Let's first show that the algorithm runs in time that is polynomial in $|X|$ and $|\mathcal{F}|$. The number of iterations of the loop in lines 4–7 is bounded above by $\min\{|X|, |\mathcal{F}|\} = O(|X| + |\mathcal{F}|)$. The loop body can be implemented to run in $O(|X| \cdot |\mathcal{F}|)$ time. Thus the algorithm runs in $O(|X| \cdot |\mathcal{F}| \cdot (|X| + |\mathcal{F}|))$ time, which is polynomial in the input size. (Exercise 35.3-3 asks for a linear-time algorithm.)

To prove the approximation bound, let $\mathcal{C}^*$ be an optimal set cover for the original instance $(X, \mathcal{F})$, and let $k = |\mathcal{C}^*|$. Since $\mathcal{C}^*$ is also a set

cover of each subset U_i of X constructed by the algorithm, we know that any subset U_i constructed by the algorithm can be covered by k sets. Therefore, if $(U_i, \mathcal{F})$ is an instance of the set-covering problem, its optimal set cover has size at most k .

If an optimal set cover for an instance $(U_i, \mathcal{F})$ has size at most k , at least one of the sets in $\mathcal{C}$ covers at least $|U_i|/k$ new elements. Thus, line 5 of GREEDY-SET-COVER, which chooses a set with the maximum number of uncovered elements, must choose a set in which the number of newly covered elements is at least $|U_i|/k$. These elements are removed when constructing U_{i+1} , giving

$$\begin{aligned} |U_{i+1}| &\leq |U_i| - |U_i|/k \\ &= |U_i| (1 - 1/k) . \end{aligned} \tag{35.8}$$

Iterating inequality (35.8) gives

$$\begin{aligned} |U_0| &= |X|, \\ |U_1| &\leq |U_0| (1 - 1/k), \\ |U_2| &\leq |U_1| (1 - 1/k) = |U| (1 - 1/k)^2, \end{aligned}$$

and in general

$$|U_i| \leq |U_0| (1 - 1/k)^i = |X| (1 - 1/k)^i . \tag{35.9}$$

The algorithm stops when $U_i = \emptyset$, which means that $|U_i| < 1$. Thus an upper bound on the number of iterations of the algorithm is the smallest value of i for which $|U_i| < 1$.

Since $1 + x \leq e^x$ for all real x (see inequality (3.14) on page 66), by letting $x = -1/k$, we have $1 - 1/k \leq e^{-1/k}$, so that $(1 - 1/k)^k \leq (e^{-1/k})^k = 1/e$. Denoting the number i of iterations by ck for some nonnegative integer c , we want c such that

$$|X| (1 - 1/k)^{ck} \leq |X| e^{-c} < 1 . \tag{35.10}$$

Multiplying both sides by e^c and then taking the natural logarithm of both sides gives $c \geq \ln |X|$, so we can choose for c any integer that is at least $\ln |X|$. We choose $c = \lceil \ln |X| \rceil$. Since $i = ck$ is an upper bound on the number of iterations, which equals the size of $\mathcal{C}$, and $k = |\mathcal{C}^*|$, we have $|\mathcal{C}| \leq i = ck = c |\mathcal{C}^*| = |\mathcal{C}^*| \lceil \ln |X| \rceil$, and the theorem follows. ■

Exercises

35.3-1

Consider each of the following words as a set of letters: {arid, dash, drain, heard, lost, nose, shun, slate, snare, thread}. Show which set cover GREEDY-SET-COVER produces when you break ties in favor of the word that appears first in the dictionary.

35.3-2

Show that the decision version of the set-covering problem is NP-complete by reducing the vertex-cover problem to it.

35.3-3

Show how to implement GREEDY-SET-COVER to run in $O(\sum_{S \in \mathcal{F}} |S|)$ time.

35.3-4

The proof of Theorem 35.4 says that when GREEDY-SET-COVER, run on the instance $(X, \mathcal{F})$, returns the subfamily $\mathcal{C}$, then $|\mathcal{C}| \leq |\mathcal{C}^*| \lceil \ln |X| \rceil$. Show that the following weaker bound is trivially true:

$$|\mathcal{C}| \leq |\mathcal{C}^*| \max \{|S| : S \in \mathcal{F}\}.$$

35.3-5

GREEDY-SET-COVER can return a number of different solutions, depending on how it breaks ties in line 5. Give a procedure BAD-SET-COVER-INSTANCE (n) that returns an n -element instance of the set-covering problem for which, depending on how line 5 breaks ties, GREEDY-SET-COVER can return a number of different solutions that is exponential in n .

35.4 Randomization and linear programming

This section studies two useful techniques for designing approximation algorithms: randomization and linear programming. It starts with a simple randomized algorithm for an optimization version of 3-CNF satisfiability, and then it shows how to design an approximation algorithm for a weighted version of the vertex-cover problem based on linear programming. This section only scratches the surface of these two powerful techniques. The chapter notes give references for further study of these areas.

A randomized approximation algorithm for MAX-3-CNF satisfiability

Just as some randomized algorithms compute exact solutions, some randomized algorithms compute approximate solutions. We say that a randomized algorithm for a problem has an *approximation ratio* of $\rho(n)$ if, for any input of size n , the *expected* cost C of the solution produced by the randomized algorithm is within a factor of $\rho(n)$ of the cost C^* of an optimal solution:

$$\max \left\{ \frac{C}{C^*}, \frac{C^*}{C} \right\} \leq \rho(n) . \quad (35.11)$$

We call a randomized algorithm that achieves an approximation ratio of $\rho(n)$ a *randomized $\rho(n)$ -approximation algorithm*. In other words, a randomized approximation algorithm is like a deterministic approximation algorithm, except that the approximation ratio is for an expected cost.

A particular instance of 3-CNF satisfiability, as defined in Section 34.4, may or may not be satisfiable. In order to be satisfiable, there must exist an assignment of the variables so that every clause evaluates to 1. If an instance is not satisfiable, you might instead want to know how “close” to satisfiable it is, that is, find an assignment of the variables that satisfies as many clauses as possible. We call the resulting maximization problem *MAX-3-CNF satisfiability*. The input to MAX-3-CNF satisfiability is the same as for 3-CNF satisfiability, and the goal is to return an assignment of the variables that maximizes the number of